

SignalReasoner: Assessing the Upper Bound of 3B Models for Signal Mathematical Reasoning

Guozheng Sun

Abstract

Post-training with supervised chain-of-thought fine-tuning and reinforcement learning from verifiable rewards has substantially improved the mathematical reasoning capabilities of large language models (LLMs). However, their application to signal processing problems remains relatively under-explored. This report investigates reinforcement fine-tuning strategies for adapting Qwen2.5-3B-Base to graduate-level signal mathematical problems from WirelessMATHBench-XL, a comprehensive benchmark for mathematical reasoning in this domain. We examine two training paradigms: (i) direct reinforcement learning (RL) on WirelessMATHBench-XL with verifiable rewards; and (ii) supervised fine-tuning (SFT) on a distilled wireless-domain chain-of-thought corpus, followed by the same domain-specific RL stage. Across both paradigms, we benchmark Group Relative Policy Optimization (GRPO), Group Sequence Policy Optimization (GSPO), and Geometric-Mean Policy Optimization (GMPO). We aim to assess whether domain-aware CoT SFT serves as an effective initialization for subsequent RL, and whether GSPO or GMPO offer advantages in stability or accuracy over GRPO for signal reasoning tasks. Our best model achieves an overall accuracy of 39.12%, representing a more than threefold improvement over the untrained Base model (12.37%).

Keywords: Signal Mathematical Reasoning, SFT, GRPO, GSPO, GMPO

Code: <https://github.com/thusunlight/SignalReasoner>

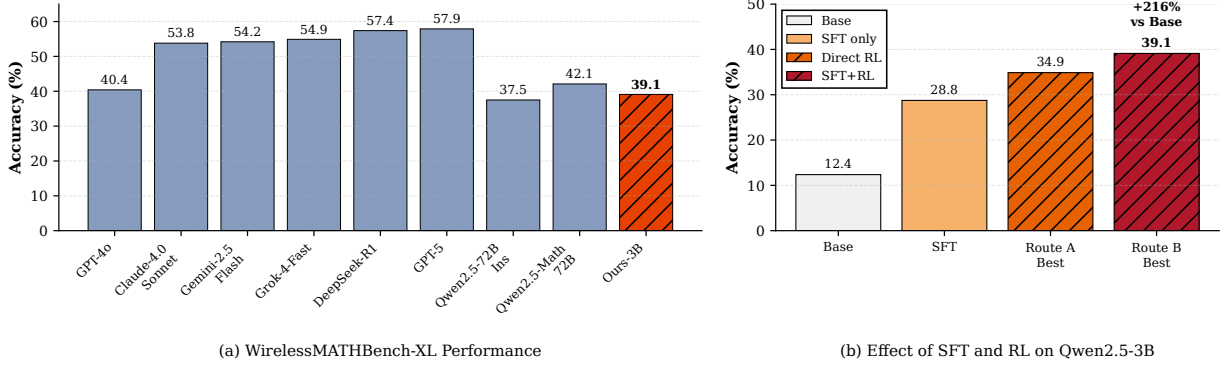


Figure 1: Teaser comparison on WirelessMATHBench-XL. Blue bars show reference strong-model results, while orange/red bars summarize our Qwen2.5-3B experiments. Route A denotes direct RL, and Route B denotes SFT+RL.

1 Introduction

Large language models (LLMs) have shown strong progress in reasoning [1], [2], [3], [4], especially when post-trained with supervised chain-of-thought data and reinforcement learning from verifiable rewards [5], [6], [7]. However, specialized engineering problems remain difficult for general-purpose models. Signal processing questions involve technologies such as MIMO and beamforming [8], RIS [9], and ISAC [10], and often combine symbolic equations, physical assumptions, and long scientific context. A correct answer may require not only algebraic manipulation, but also recognition of channel models, signal dimensions, and the meaning of variables inside a system equation. Existing sub-10B models struggle with such tasks, as they lack the capacity to jointly handle domain-specific knowledge and multi-step mathematical derivations [11], [12], [13].

This report investigates whether Qwen2.5-3B [14] can be adapted to this domain through reinforcement fine-tuning, and to what extent post-training improves its reasoning capability. We conduct experiments on WirelessMATHBench-XL [12], which consists of graduate-level signal mathematical problems derived from technical literature, and evaluate two training paradigms with three RL algorithms. In the first route, Qwen2.5-3B-Base is directly optimized with RL on WirelessMATHBench-XL. This route tests whether the base model can acquire domain reasoning behavior from reward signals alone. In the second route, Qwen2.5-3B-Base is first fine-tuned on Wireless-CoT-Mix, a 3,542-example mixture of distilled wireless-domain CoT trajectories and NuminaMath-CoT [15] samples, and then further optimized with RL on WirelessMATHBench-XL. This route tests whether domain-aware CoT SFT provides a better initialization for subsequent signal-domain RL.

For the RL stage, we compare three group-based policy optimization algorithms, namely GRPO [6], GSPO [16], and GMPO [17]. GRPO removes the need for a separate critic by estimating advantages from multiple responses sampled for the same prompt. GSPO changes the optimization unit from tokens to full sequences, aiming to improve training stability by aligning sequence-level rewards with sequence-level clipping. GMPO instead stabilizes token-level updates by replacing arithmetic aggregation with a geometric-mean objective that reduces sensitivity to outlier importance ratios. Experiment results show that our best model achieves an overall accuracy of 39.12%, representing a more than threefold improvement over the untrained Base model (12.37%).

Contributions. This report makes the following contributions:

- We conduct a systematic empirical comparison of two fine-tuning routes for adapting Qwen2.5-3B to signal processing tasks on WirelessMATHBench-XL, and demonstrate that RL consistently improves overall accuracy from 12% to 39%.
- We benchmark GRPO, GSPO, and GMPO across both routes, finding that GSPO and GMPO outperform GRPO in accuracy, convergence speed, and output token efficiency, with GMPO after SFT achieving the best overall result.
- We identify a reasoning-shortening phenomenon under GSPO and GMPO, where the model learns to produce compact answers with reduced chain-of-thought detail, and propose a plausible explanation for the underlying mechanisms of this behavior.

2 Related Work

2.1 Reinforcement Learning for Reasoning

Aligning language models with human intent has driven the RLHF paradigm, in which a reward model trained on human preferences guides PPO-based policy optimization [18], [19]. DPO [20] later simplified this by eliminating the reward model, but its formulation is tied to pairwise preference data. It does not accommodate verifiable rewards such as mathematical correctness or code execution results, which are deterministically computed from model outputs. This distinction motivates a separate line of work on reward-driven RL, where the policy is optimized directly against task-specific, automatically evaluated reward functions.

Mathematical reasoning is a natural domain for such methods. STaR [5] bootstrapped reasoning via iterative self-training on correct rationales. DeepSeekMath [6] introduced GRPO and DeepSeek-R1 [7] later demonstrated that large-scale RL with a simple rule-based reward can elicit sophisticated reasoning behaviors. DAPO [21] proposed a Dynamic sAmpling Policy Optimization algorithm and introduced four key techniques to stabilize RL in the long chain-of-thought setting, including Clip-Higher for wider exploration and dynamic sampling to filter out low-quality prompts. GSPO [16] and GMPO [17], the two variants directly evaluated in this report, address sequence-level versus token-level optimization and arithmetic-mean versus geometric-mean aggregation, respectively.

2.2 Chain-of-Thought Reasoning

Chain-of-thought (CoT) prompting [1] elicits step-by-step reasoning from LLMs by augmenting the input with intermediate reasoning traces, substantially improving performance on multi-step arithmetic, and commonsense tasks. Kojima et al. [2] later showed that the same effect can be achieved without exemplars, simply through the prompt “Let’s think step by step”, suggesting that structured reasoning is an inherent capability that can be triggered rather than taught. Self-consistency [3] further improves reliability by sampling multiple reasoning paths and marginalizing over them via majority voting. This exploits the observation that correct reasoning tends to be more consistent across diverse samples than incorrect reasoning.

Beyond prompting, CoT data has become central to post-training. Large-scale datasets such as MetaMathQA [22], OpenMathInstruct-1 [23], and NuminaMath-CoT [15] provide large collections of problem–solution pairs with CoT-format rationales, enabling SFT to instill structured step-by-step reasoning as a behavioral prior. In this report, we further distill wireless-specific CoT trajectories from DeepSeek-V3 to align this reasoning prior with the target signal-processing domain. In the RL stage, this prior shapes the model’s exploration, since responses already follow a derivation-then-answer format and the verifiable reward can focus on correctness rather than format discovery. The interaction between this CoT initialization and subsequent RL is a key axis of the present study.

2.3 LLMs in Signal Reasoning

Signal processing and wireless communications impose stringent requirements on mathematical precision, particularly for tasks such as channel estimation, interference management, and beam-forming [24], [25]. Some preliminary works have explored the use of LLMs in wireless contexts, focusing on domain-specific knowledge extraction and basic recall of technical standards [26], [27], [28], [29]. Notably, TelecomGPT [30] has extended LLM capabilities to higher-level tasks such as wireless-specific code generation and formula completion. However, these early works primarily emphasize knowledge retrieval or summarization, without systematically evaluating whether LLMs can perform the multi-step mathematical reasoning required in actual signal processing engineering systems.

WirelessMathBench [11] took a step by introducing a curated benchmark of 587 expert-level signal processing math problems. Its successor WirelessMATHBench-XL [12] scaled the dataset to 4,027 problems and demonstrated that compact models fine-tuned with GRPO can approach the performance of much larger general-purpose models. Building on this line of work, the present report systematically compares several reinforcement fine-tuning strategies for adapting a sub-10B model to signal-domain mathematical reasoning.

3 Method

3.1 Supervised Fine-Tuning

Given a supervised dataset $\mathcal{D}_{\text{SFT}} = \{(x, y)\}$, SFT minimizes the autoregressive negative log-likelihood:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{SFT}}} \frac{1}{|y|} \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t \mid x, y_{<t}). \quad (1)$$

In this project, SFT is applied only in Route B using Wireless-CoT-Mix. The purpose is to initialize the model with both signal-domain reasoning trajectories and general mathematical CoT behavior: step decomposition, symbolic manipulation, and final-answer formatting.

3.2 Group Relative Policy Optimization

Proximal Policy Optimization (PPO) [19] has become the dominant RL algorithm for LLM post-training, owing to its stable clipped objective and compatibility with the autoregressive token-generation paradigm. However, PPO relies on a separate critic network to estimate per-token state values, which are then combined with empirical returns via Generalized Advantage Estimation (GAE) to produce low-variance advantage signals. Training the critic alongside the

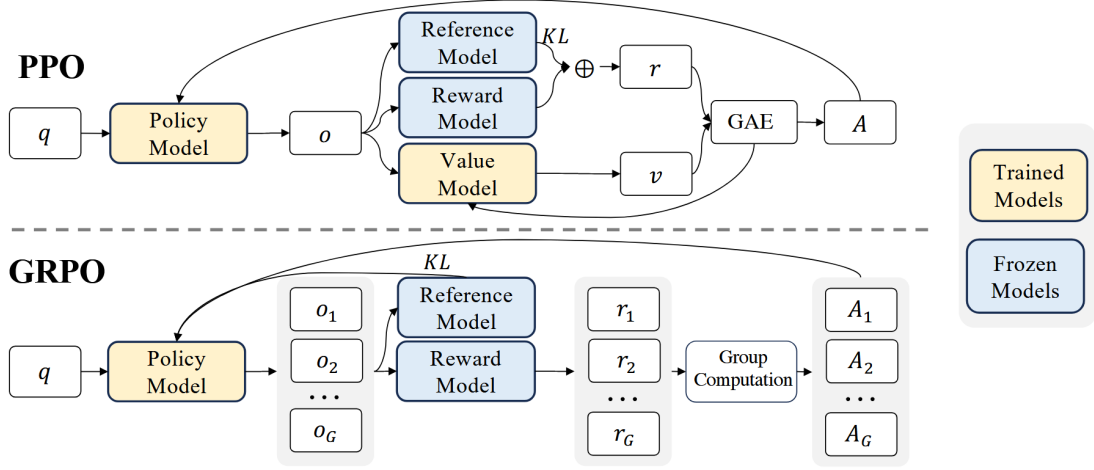


Figure 2: Comparison of PPO and GRPO architectures, adapted from DeepSeek-R1 [7]. PPO requires a separate critic network for advantage estimation via GAE, while GRPO eliminates the critic by computing group-relative advantages from multiple sampled responses.

policy approximately doubles GPU memory consumption and inaccurate critic can bias the advantage and misdirect policy updates.

GRPO, proposed in DeepSeekMath [6] and later adopted in DeepSeek-R1 [7], sidesteps the critic entirely. For each prompt x , GRPO samples a group of G responses $\{y_i\}_{i=1}^G$ from the old policy $\pi_{\theta_{\text{old}}}$ and normalizes the reward within the group to obtain a baseline-free advantage:

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_j\}_{j=1}^G)}{\text{std}(\{r_j\}_{j=1}^G) + \epsilon}. \quad (2)$$

GRPO maximizes a clipped token-level surrogate with a KL penalty that regularizes the current policy towards a frozen reference π_{ref} :

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right], \quad (3)$$

where $\rho_{i,t}(\theta) = \pi_{\theta}(y_{i,t} \mid x, y_{i,<t}) / \pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})$ is the token-level importance ratio and β controls the KL penalty strength. The KL divergence is estimated by the unbiased, guaranteed non-negative estimator

$$D_{\text{KL}}^{(t)} = \frac{\pi_{\text{ref}}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})} - \log \frac{\pi_{\text{ref}}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})} - 1. \quad (4)$$

All tokens in a response share the same advantage $\hat{A}_i$, since only the final token typically receives a reward. This design offers three key benefits. It halves memory usage by removing the critic, which is essential when training on consumer or mid-range GPUs. The group-relative normalization is invariant to the scale and shift of the reward function, so the same clipping hyperparameters transfer across tasks with different reward ranges without manual tuning. And the group mean acts as an implicit baseline, providing variance reduction comparable to a learned value function at no extra cost. These properties make GRPO particularly well-suited for reasoning benchmarks like WirelessMATHBench-XL, where a clean verifiable reward signal is available for each complete response.

3.3 Group Sequence Policy Optimization

In large-scale RL training, a big rollout batch is typically partitioned into mini-batches for gradient updates to maximize hardware utilization. This introduces an off-policy setting where responses are sampled from an old policy $\pi_{\theta_{\text{old}}}$ rather than the current policy π_{θ} , which is why clipping mechanisms

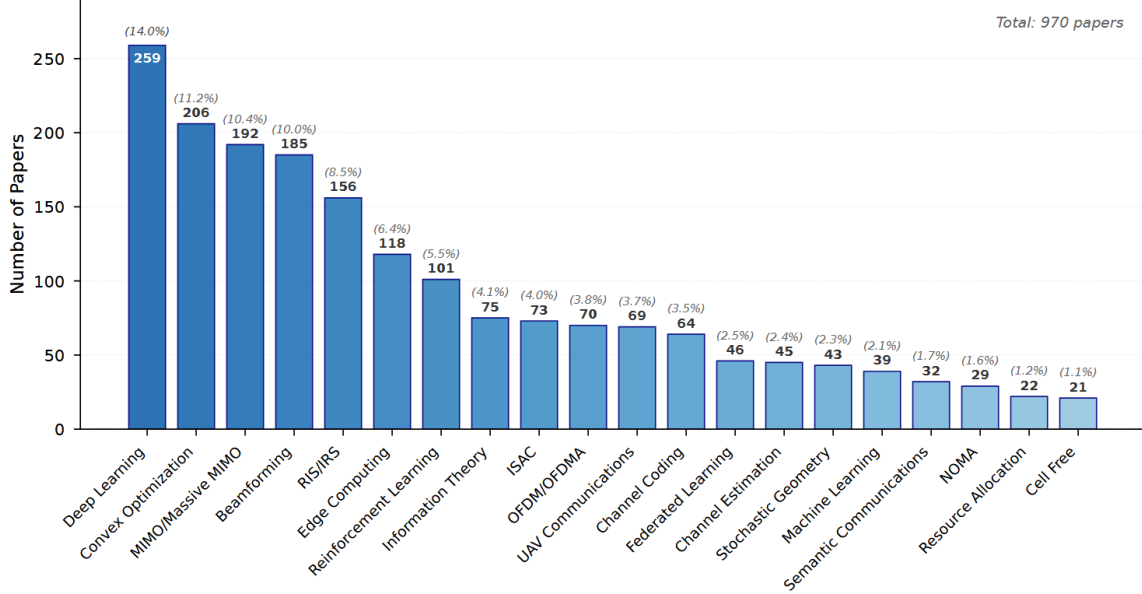


Figure 3: Distribution of the top 20 key techniques across the 970 source papers in WirelessMATHBench-XL [12].

are employed in PPO and GRPO. However, GSPO identifies a more fundamental issue in GRPO: its token-level importance sampling is ill-posed. Importance sampling estimates an expectation under a target distribution by averaging re-weighted samples drawn from a behavior distribution, and the correction relies on aggregating over multiple samples from the same distribution. GRPO applies the importance weight $\rho_{i,t}(\theta)$ at each token based on a single draw from the next-token distribution, which fails to perform meaningful distribution correction and instead injects high-variance noise into the gradient. This noise accumulates over long sequences and can lead to irreversible model collapse.

The root cause points to a simple principle that the unit of optimization should match the unit of reward. Since the reward is assigned to an entire response, applying off-policy correction and clipping at the token level is fundamentally mismatched. GSPO addresses this by shifting the optimization unit from individual tokens to complete sequences. The sequence-level importance ratio is length-normalized to unify the numerical range across responses of different lengths:

$$s_i(\theta) = \left[\frac{\pi_\theta(y_i | x)}{\pi_{\theta_{\text{old}}}(y_i | x)} \right]^{1/|y_i|} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \rho_{i,t}(\theta) \right). \quad (5)$$

The clipping are then applied at the sequence level:

$$\mathcal{J}_{\text{GSPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right) \right]. \quad (6)$$

By operating at the sequence level, GSPO aligns the optimization unit with the reward unit and mitigates the training collapse caused by token-level importance sampling noise.

3.4 Geometric-Mean Policy Optimization

In GRPO, the per-token importance ratios $\rho_{i,t}(\theta)$ are averaged arithmetically across the response. While simple, this arithmetic mean is highly sensitive to outliers: a single token with an extreme ratio (e.g., where the current policy assigns dramatically higher or lower probability than the old policy) can dominate the entire average, especially in long responses. GMPO addresses this by replacing the arithmetic mean with a geometric mean. By the AM–GM inequality, $|\mathcal{J}_{\text{GMPO}}| \leq |\mathcal{J}_{\text{GRPO}}|$, which gives GMPO a strictly narrower value range and lower optimization

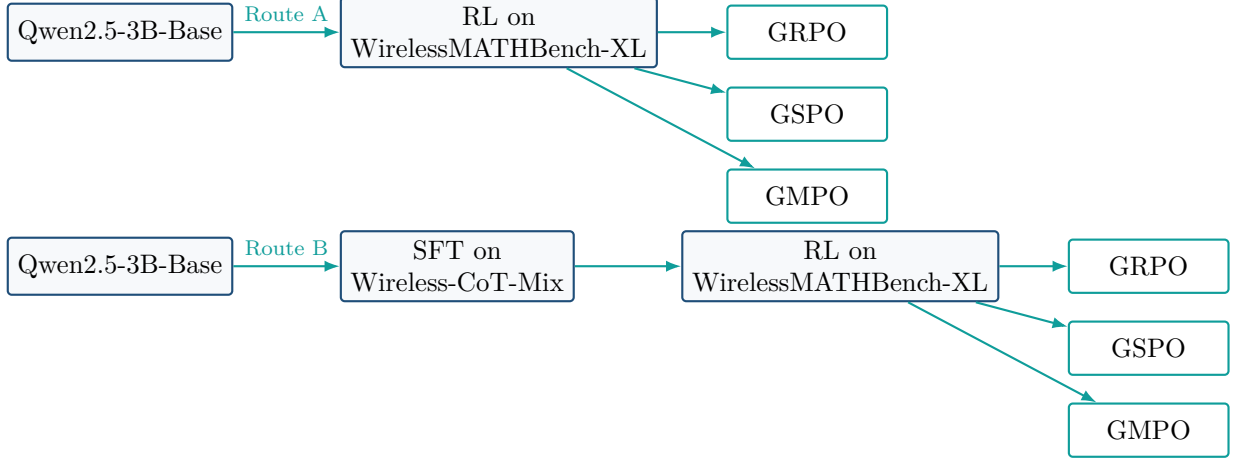


Figure 4: Two-route training design. Route A applies RL directly to Qwen2.5-3B-Base. Route B first performs SFT on Wireless-CoT-Mix and then applies signal-domain RL.

variance. The geometric mean effectively dampens the influence of any single token, requiring a more uniform consensus across the sequence for the update to be large.

A further difference from GSPO concerns the clipping granularity. GSPO clips at the sequence level, which can discard the entire gradient signal for a response when its sequence-level ratio exceeds the clipping threshold. GMPO instead clips individual token-level ratios in log-space before geometric aggregation. This preserves gradient signals for unclipped tokens while only suppressing the extreme ones, making more efficient use of the sampled data. Inspired by DAPO’s Clip-Higher strategy [21], GMPO adopts a wider clipping range in the log-ratio space, i.e., $\rho_{i,t}^{\text{sgn}(\hat{A}_i)} \in [e^{-0.4}, e^{0.4}]$, to improve exploration while maintaining stable optimization. The simplified (unclipped) GMPO objective is

$$\mathcal{J}_{\text{GMPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \left(\prod_{t=1}^{|y_i|} \frac{\pi_{\theta}(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | x, y_{i,<t})} \left| \hat{A}_i \right| \right)^{\frac{1}{|y_i|}} \text{sgn}(\hat{A}_i) \right]. \quad (7)$$

For the full objective, token-level clipping is applied:

$$\mathcal{J}_{\text{GMPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \left\{ \prod_{t=1}^{|y_i|} \min(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), \varepsilon_{\text{low}}, \varepsilon_{\text{high}}) \hat{A}_i) \right\}^{\frac{1}{|y_i|}} \cdot \text{sgn}(\hat{A}_i) \right], \quad (8)$$

where $\text{sgn}(\hat{A}_i)$ returns $+1$ for positive advantages and -1 otherwise, and $(\varepsilon_{\text{low}}, \varepsilon_{\text{high}})$ are the lower and upper clipping thresholds on the importance ratio (e.g., $e^{-0.4}$ and $e^{0.4}$, following DAPO’s Clip-Higher strategy). Unlike GSPO, which clips at the sequence level, GMPO clips individual token-level ratios, preserving gradient signals for unclipped tokens while suppressing extreme outliers. In our experiments, GMPO is evaluated as a stability-oriented alternative to GRPO under the same prompts, reward function, and base model.

4 Task and Training Routes

4.1 Benchmark Task and Dataset

WirelessMATHBench-XL [12] contains 4,027 graduate-level signal processing mathematical problems curated from 970 technical papers. The dataset spans a broad range of topics, including MIMO, RIS, ISAC, beamforming, NOMA, satellite communications, and so on. The problems are organized into two question types: multiple-choice questions and fill-in-the-blank questions. The dataset is split into 3,227 training examples and 800 test examples. Each sample contains a prompt x assembled from structured fields, including a technical background passage, a question text, an optional equation, and optional answer choices, and a gold answer a . The model generates a response y that should contain a concise reasoning process and a final answer in `\boxed{}`.

Table 1: Main experimental methods.

ID	Route	Initialization	RL Algorithm
Base	/	Base	/
A-GRPO	RL	Base	GRPO
A-GSPO	RL	Base	GSPO
A-GMPO	RL	Base	GMPO
Base-SFT	SFT	Base + SFT	/
B-GRPO	SFT + RL	Base + SFT	GRPO
B-GSPO	SFT + RL	Base + SFT	GSPO
B-GMPO	SFT + RL	Base + SFT	GMPO

Wireless-CoT-Mix is the SFT corpus constructed for Route B. It contains 3,542 examples, including 1,546 wireless-domain question–reasoning pairs and 1,996 general-domain examples sampled from NuminaMath-CoT [15]. The wireless portion is distilled from DeepSeek-V3 using two sources: WirelessMATHBench-XL, whose official split contains 3,227 training and 800 test instances, and WirelessMathBench [11], which contains 587 instances. For WirelessMATHBench-XL, we sample approximately 50% of the training split for SFT construction, corresponding to about 1,613 candidate problems, and obtain 1,503 verified CoT trajectories after generation and answer checking. For WirelessMathBench, the same pipeline produces 483 verified CoT trajectories. After length filtering, the final wireless SFT subset contains 1,546 usable question–reasoning pairs.

4.2 Route A: Direct RL from Qwen2.5-3B-Base

As shown in Figure 4, the first route starts from the raw Qwen2.5-3B-Base checkpoint and directly performs RL on WirelessMATHBench-XL. This route is designed to answer the following question:

Can signal-domain verifiable rewards alone induce useful mathematical reasoning behavior in a compact base model?

Under this route, we compare three group-based RL algorithms, namely GRPO, GSPO, and GMPO, to evaluate their effectiveness when applied directly to a base model without any SFT initialization.

4.3 Route B: SFT on Wireless-CoT-Mix Followed by RL

As shown in Figure 4, the second route first fine-tunes Qwen2.5-3B-Base on Wireless-CoT-Mix, then applies RL on WirelessMATHBench-XL. This route is designed to answer the following question:

Does domain-aware CoT SFT provide a better initialization for subsequent signal-domain RL?

The SFT stage is intended to initialize the model with wireless-domain reasoning patterns, mathematical decomposition habits, and final-answer discipline while preserving general mathematical reasoning through the NuminaMath-CoT mixture. The subsequent RL stage then further specializes this prior toward signal-processing tasks with verifiable rewards. As in Route A, we compare GRPO, GSPO, and GMPO at the RL stage.

5 Experimental Setup

5.1 Compared Methods

The eight compared methods are listed in Table 1, comprising two baselines (Base and Base-SFT) and six RL variants (A-GRPO, A-GSPO, A-GMPO, B-GRPO, B-GSPO, B-GMPO). This design isolates two questions: whether SFT improves the RL starting point, and which RL algorithm is most effective under each starting point.

Table 2: Performance of SignalReasoner models on the WirelessMATHBench-XL test set (%). MCQ: Multiple Choice Questions, Fill-in: Fill-in-the-blank, FEC: Full Equation Completion.

Model	Size	MCQ	Fill-in	FEC	Overall	Output Tokens
<i>SignalReasoner 3B Models</i>						
Base	3B	28.57	9.03	9.42	12.37	907.41
Base-SFT	3B	42.11	26.68	24.61	28.75	604.45
A-GRPO	3B	57.89	30.88	27.23	34.50	668.33
A-GSPO	3B	61.65	30.88	26.18	34.88	279.25
A-GMPO	3B	51.88	32.98	26.70	34.63	395.84
B-GRPO	3B	49.62	33.61	25.65	34.38	600.10
B-GSPO	3B	57.89	35.08	30.37	37.75	243.59
B-GMPO	3B	54.14	38.03	31.41	39.12	258.87

5.2 Hyperparameter Settings

All experiments are conducted using the open-source VeRL framework¹. The SFT stage trains Qwen2.5-3B-Base on Wireless-CoT-Mix for 1 epoch with a learning rate of 3×10^{-6} and a batch size of 36. The RL stage uses a group size of 4, a learning rate of 1×10^{-6} , and a batch size of 18, training for 1,000 steps on three NVIDIA RTX 4090 GPUs. The maximum response length is set to 768 tokens for both SFT training and RL generation. Detailed hyperparameters are provided in Table A1 in the appendix.

5.3 Reward Design

The reward function extracts all `\boxed{}` expressions from the model’s response y and compares them element-wise against the ground-truth answer a . Let M denote the number of ground-truth boxed expressions and m the number of extracted expressions that match the ground truth exactly. The reward is defined as

$$R(y, a) = \begin{cases} 0.0, & \text{no } \boxed{} \text{ detected,} \\ 0.1, & m = 0, \\ 0.2, & 0 < m < M, \\ 1.0, & m = M. \end{cases} \quad (9)$$

6 Experimental Results

6.1 Main Results

Table 2 reports the main comparison. RL training consistently improves performance, with overall accuracy rising from 12.37% for the Base model to 34–39% after reinforcement fine-tuning. In addition, SFT alone on Wireless-CoT-Mix substantially improves overall accuracy to 28.75%, showing that the distilled wireless-domain CoT data injects useful signal-specific reasoning behavior before RL. SFT also reduces the average output length from 907.41 to 604.45 tokens, suggesting that the mixed CoT corpus teaches a more structured and concise response format. Meanwhile, Route B achieves the highest overall accuracy. B-GMPO reaches 39.12% and B-GSPO reaches 37.75%, suggesting that SFT initialization provides a clear benefit when combined with sequence-level or geometric-mean optimization. In contrast, for GRPO, direct RL and the SFT-pretrained variant remain close, implying that GRPO’s token-level advantage estimation may be less sensitive to the quality of the initial policy.

Meanwhile, a striking pattern emerges in output token length. GRPO variants produce 545–668 tokens on average, maintaining substantial reasoning traces. In contrast, GSPO and GMPO

¹<https://github.com/volcengine/verl>

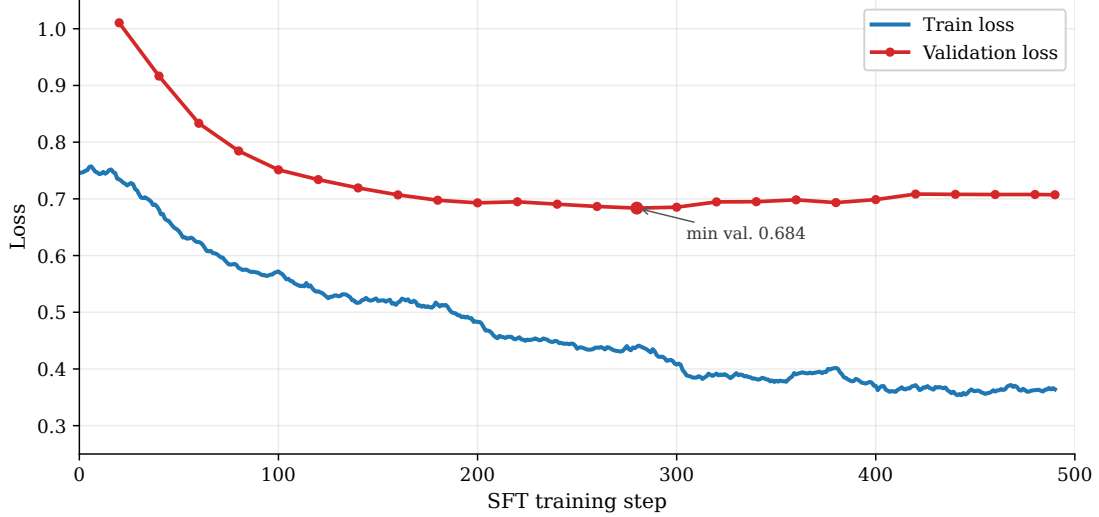


Figure 5: SFT loss curves on Wireless-CoT-Mix. The blue curve shows smoothed training loss, and the red curve shows validation loss evaluated every 20 steps.

drastically reduce output length. A-GSPO and A-GMPO produce 279.25 and 395.84 tokens respectively, while B-GSPO and B-GMPO further reduce the average length to 243.59 and 258.87 tokens. At this token budget, the model can still provide concise derivations, but the reasoning traces are much shorter than those produced by GRPO. We attribute this phenomenon to two compounding effects. First, both GSPO and GMPO implicitly penalize long sequences. GSPO computes a single importance ratio per sequence and applies the same update magnitude to every token regardless of length. A long incorrect response, which is common during early exploration, therefore receives the same per-sequence penalty as a short incorrect one but distributes that penalty across more tokens, resulting in a stronger cumulative suppression of long outputs over training. GMPO’s geometric-mean objective suppresses outlier token-level ratios, which disproportionately arise in long reasoning chains with high variance, thereby reducing the effective update signal for lengthy responses. Second, SFT pre-training on Wireless-CoT-Mix already biases the model toward a structured but concise output format. When this compact prior is combined with the implicit length penalty of GSPO and GMPO, the RL optimization may discover a shortcut, namely outputting the answer with little reasoning, since the verifiable reward only checks the final `\boxed{}` expression and does not reward intermediate steps. The result is a “reasoning shortening” effect in which the model achieves competitive or even superior accuracy with dramatically fewer tokens, at the cost of less detailed step-by-step derivations. Whether this shortening is desirable depends on the deployment context. It improves inference efficiency but eliminates the auditability and error-diagnosis benefits of explicit reasoning traces.

6.2 SFT Training Dynamics

Figure 5 shows the training and validation loss curves during the SFT stage on Wireless-CoT-Mix. The smoothed training loss decreases from roughly 0.75 in the early steps to about 0.36 by the end of training, indicating stable optimization on the mixed wireless and general-domain CoT corpus. The validation loss drops rapidly from 1.01 to a minimum of approximately 0.684, after which it plateaus and slightly increases to around 0.71. This pattern suggests that the model quickly learns the supervised CoT format and domain-specific answer structure, while later updates mainly continue fitting the training distribution rather than improving validation loss. Empirically, however, we found that using the final checkpoint as the initialization for RL yields noticeably worse downstream accuracy compared to an earlier checkpoint. We attribute this to over-training on the SFT distribution. By the end of training, the model’s output distribution can become overly concentrated on the specific formatting and answer patterns of Wireless-CoT-Mix, reducing policy entropy and limiting the exploration capacity needed for RL to adapt to the signal domain.

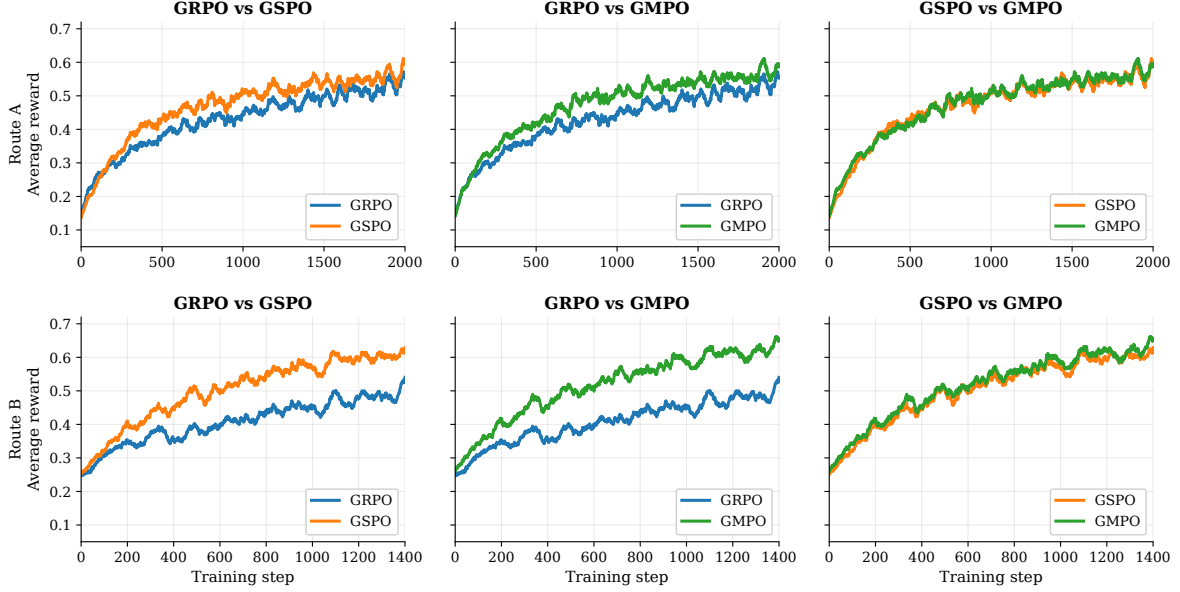


Figure 6: Algorithm comparisons within each route. Top row: direct RL from the Base model (Route A), truncated at 2,000 steps. Bottom row: the SFT+RL route, where RL starts after supervised training on Wireless-CoT-Mix (Route B), truncated at 1,400 steps.

The excessively narrow prior effectively traps the RL optimizer in a local region of the policy space from which it is difficult to escape. Based on this observation, all Route B experiments in this report use an intermediate SFT checkpoint, which balances structured reasoning capability with sufficient policy diversity for effective RL fine-tuning.

6.3 RL Training Dynamics

Figure 6 and Figure 7 show the reward curves for all six RL variants over the available training steps. For the SFT+RL route, the plotted trajectories correspond to RL runs that start from the intermediate Wireless-CoT-Mix checkpoint described above, rather than from the raw Base model. Thus, the bottom row of Figure 6 should be read as the second-stage optimization behavior after supervised CoT distillation: SFT first establishes a wireless-aware reasoning prior, and RL then refines the policy with the same verifiable reward used in Route A. Figure 6 compares algorithm pairs under the same initialization. Under the Base route, GSPO and GMPO exhibit more stable reward improvement than GRPO, which shows larger oscillations particularly in the early phase. GSPO’s sequence-level updates produce the smoothest curves, consistent with its design of reducing per-token variance. Under the SFT route, the gap between algorithms narrows. GSPO and GMPO still maintain slightly lower variance than GRPO. Notably, GMPO achieves the highest final reward under both routes, corroborating its top overall accuracy in Table 2. Moreover, both GSPO and GMPO exhibit markedly faster convergence than GRPO. Their reward curves rise more steeply in the first few hundred steps and reach a higher plateau earlier, indicating superior training efficiency.

Figure 7 compares Route A and Route B for each algorithm. For GRPO, the B-GRPO shows a modest advantage in early-step reward growth but the two routes converge to similar levels by the end of training, consistent with the small accuracy gap observed in Table 2. For GSPO and GMPO, the SFT-initialized variants achieve higher reward levels more rapidly than their Base counterparts, indicating that the structured CoT prior from SFT interacts synergistically with sequence-level and geometric-mean optimization. This acceleration is most pronounced for GMPO, where B-GMPO consistently leads A-GMPO throughout training and ultimately delivers the highest accuracy.

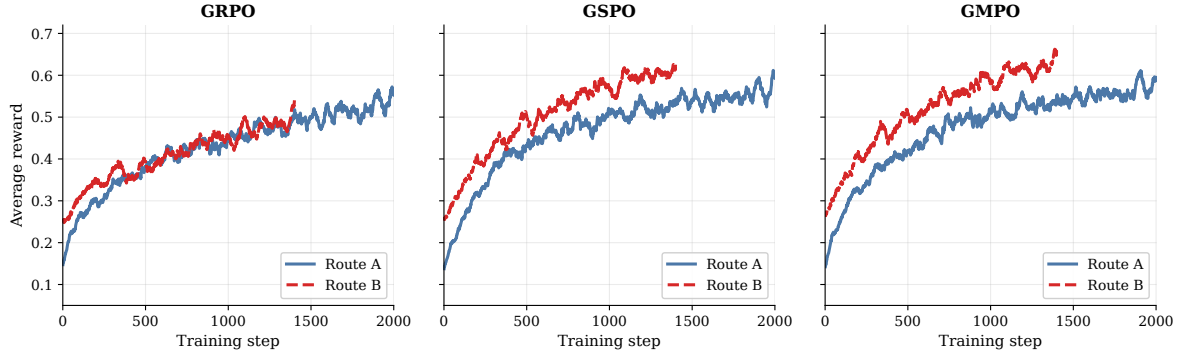


Figure 7: Route comparisons per algorithm. Solid lines denote Route A, direct RL from the Base model, truncated at 2,000 steps. Dashed lines denote Route B, the SFT+RL route, truncated at 1,400 steps.

7 Conclusion

This report studied reinforcement fine-tuning of Qwen2.5-3B-Base for signal mathematical reasoning on WirelessMATHBench-XL, comparing direct RL with an SFT-then-RL route under GRPO, GSPO, and GMPO. The results show that both stages contribute: SFT alone improves overall accuracy from 12.37% for the Base model to 28.75%, while subsequent RL reaches 39.12% with B-GMPO. The SFT-then-RL route is particularly effective for GSPO and GMPO, with B-GSPO and B-GMPO achieving 37.75% and 39.12%, respectively. The advantage is less pronounced for GRPO, whose direct-RL and SFT-then-RL variants obtain similar overall scores. Thus, SFT provides a useful domain-specific starting point, but its benefit depends on the RL objective.

Across the three algorithms, GSPO and GMPO show the clearest gains over GRPO in the SFT-then-RL setting, together with faster early reward growth. They also produce much shorter responses: B-GSPO and B-GMPO average 243.59 and 258.87 output tokens, compared with 600.10 for B-GRPO. This shortening improves output efficiency, but it should not be interpreted as evidence that the model generates more complete reasoning. Because the current verifiable reward primarily checks the final boxed answer, optimization can favor concise responses with limited intermediate derivation. The findings therefore expose a central limitation of answer-only rewards for small reasoning models: accuracy and efficiency can improve while the transparency of the reasoning trace decreases. Future work should combine answer verification with process-sensitive rewards or auxiliary supervision that explicitly preserves useful intermediate reasoning.

References

- [1] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.
- [2] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 22 199–22 213, 2022.
- [3] X. Wang et al., “Self-consistency improves chain of thought reasoning in language models,” *International Conference on Learning Representations*, 2023.
- [4] X. Gong, G. Sun, P. Xu, and Y. Mu, “RePlan-Bot: Multi-level replanning for embodied instruction following,” *arXiv preprint arXiv:2605.25851*, 2026. arXiv: 2605.25851. [Online]. Available: <https://arxiv.org/abs/2605.25851>
- [5] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, “STaR: Bootstrapping reasoning with reasoning,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 15 476–15 488, 2022.
- [6] Z. Shao et al., “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024. arXiv: 2402.03300. [Online]. Available: <https://arxiv.org/abs/2402.03300>
- [7] D. Guo et al., “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025. arXiv: 2501.12948. [Online]. Available: <https://arxiv.org/abs/2501.12948>
- [8] A. F. Molisch, *Wireless Communications: From Fundamentals to Beyond 5G*, 3rd ed. Wiley, 2022.
- [9] E. Basar, M. Di Renzo, J. De Rosny, M. Debbah, M.-S. Alouini, and R. Zhang, “Wireless communications through reconfigurable intelligent surfaces,” *IEEE Access*, vol. 7, pp. 116 753–116 773, 2019.
- [10] A. Liu et al., “A survey on fundamental limits of integrated sensing and communication,” *IEEE Communications Surveys & Tutorials*, vol. 24, no. 2, pp. 994–1034, 2022.
- [11] X. Li, M. Liu, L. Wei, J. An, M. Debbah, and C. Yuen, “WirelessMathBench: A mathematical modeling benchmark for LLMs in wireless communications,” *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.
- [12] X. Li et al., “WirelessMathLM: Teaching mathematical reasoning for LLMs in wireless communications with reinforcement learning,” *arXiv preprint arXiv:2509.23219*, 2025. arXiv: 2509.23219. [Online]. Available: <https://arxiv.org/abs/2509.23219>
- [13] J. Wei et al., “Emergent abilities of large language models,” *Transactions on Machine Learning Research*, 2022.
- [14] A. Yang et al., “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2025.
- [15] AI-MO et al., *NuminaMath: A collection of mathematical reasoning datasets*, Hugging Face repository, 2024. [Online]. Available: <https://huggingface.co/datasets/AI-MO/NuminaMath-CoT>
- [16] C. Zheng et al., “Group sequence policy optimization,” *arXiv preprint arXiv:2507.18071*, 2025. arXiv: 2507.18071. [Online]. Available: <https://arxiv.org/abs/2507.18071>
- [17] Y. Zhao et al., “Geometric-mean policy optimization,” *arXiv preprint arXiv:2507.20673*, 2025. arXiv: 2507.20673. [Online]. Available: <https://arxiv.org/abs/2507.20673>
- [18] L. Ouyang et al., “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.
- [19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

- [20] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” *Advances in Neural Information Processing Systems*, 2023.
- [21] Q. Yu et al., “DAPO: An open-source LLM reinforcement learning system at scale,” *arXiv preprint arXiv:2503.14476*, 2025. arXiv: 2503.14476. [Online]. Available: <https://arxiv.org/abs/2503.14476>
- [22] L. Yu et al., “MetaMath: Bootstrap your own mathematical questions for large language models,” *arXiv preprint arXiv:2309.12284*, 2023.
- [23] S. Toshniwal, I. Moshkov, S. Narenthiran, D. Gitman, F. Jia, and I. Gitman, “OpenMathInstruct-1: A 1.8 million math instruction tuning dataset,” *Advances in Neural Information Processing Systems*, 2024.
- [24] V. R. Cadambe and S. A. Jafar, “Interference alignment and degrees of freedom of the K-user interference channel,” *IEEE Transactions on Information Theory*, vol. 54, no. 8, pp. 3425–3441, 2008.
- [25] D. Gesbert, S. Hanly, H. Huang, S. Shamai Shitz, O. Simeone, and W. Yu, “Multi-cell MIMO cooperative networks: A new look at interference,” *IEEE Journal on Selected Areas in Communications*, vol. 28, no. 9, pp. 1380–1408, 2010.
- [26] A. Maatouk, N. Piovesan, F. Ayed, A. De Domenico, and M. Debbah, “Large language models for telecom: Forthcoming impact on the industry,” *IEEE Communications Magazine*, 2024.
- [27] A. Maatouk, K. C. Ampudia, R. Ying, and L. Tassiulas, “Tele-LLMs: A series of specialized large language models for telecommunications,” *IEEE Access*, 2024.
- [28] V. Colle, M. Sana, N. Piovesan, A. De Domenico, F. Ayed, and M. Debbah, “TeleMath: A benchmark for large language models in telecom mathematical problem solving,” *arXiv preprint arXiv:2506.10674*, 2025.
- [29] J. Tong et al., “WirelessBench: A tolerance-aware LLM agent benchmark for wireless network intelligence,” *arXiv preprint arXiv:2603.21251*, 2026.
- [30] H. Zou et al., “TelecomGPT: A framework to build telecom-specific large language models,” *arXiv preprint arXiv:2407.09424*, 2024.

Appendix

A Data and Model Preprocessing

A.1 SFT Data

The SFT stage uses Wireless-CoT-Mix, a 3,542-example corpus constructed from both wireless-domain and general-domain mathematical reasoning data. The wireless portion is obtained by distilling CoT trajectories from DeepSeek-V3. We use two sources: WirelessMATHBench-XL, which contains 3,227 training and 800 test instances, and WirelessMathBench, which contains 587 instances. For WirelessMATHBench-XL, we sample approximately half of the training split for SFT construction, yielding about 1,613 candidate problems. The distillation and verification pipeline produces 1,503 valid CoT trajectories from this subset and 483 valid CoT trajectories from WirelessMathBench.

For each candidate problem, the prompt includes the domain background, question, masked equation, and ground-truth answer. DeepSeek-V3 is asked to generate concise step-by-step reasoning that naturally reaches the provided answer and ends with the final result enclosed in `\boxed{}`. We then verify that every generated `\boxed{}` expression exactly matches the corresponding ground-truth answer. Failed generations are retried up to five times and discarded if they still fail verification. The generated reasoning is restricted to 512 tokens, and the sampling temperature is set to 1.0.

After answer verification and length filtering, the wireless SFT subset contains 1,546 question–reasoning pairs. To preserve general mathematical reasoning ability, we add 1,996 examples sampled from NuminaMath-CoT [15], yielding the final 3,542-example SFT corpus. Each instance is reformatted as a two-turn conversation: a user message containing the problem plus a “reason step by step” instruction, and an assistant message containing the distilled reasoning. The resulting data is saved in Parquet format for efficient loading during training.

A.2 RL Data

The RL stage uses the WirelessMATHBench-XL training and test splits. We identified a critical preprocessing issue for multiple-choice (MCQ) samples: in the original dataset, the ground-truth answer for MCQ problems is stored as a raw letter (e.g., A, B, C, or D) rather than in `\boxed{}` format, causing the verifiable reward function to fail to match answers during RL training. To address this, we preprocess both the training and test sets by wrapping bare-letter MCQ ground-truth answers in `\boxed{}`. Both datasets are stored in Parquet format for compatibility with the VeRL training framework.

A.3 Model Configuration

A subtle but critical issue arises when using Qwen2.5-3B-Base directly for SFT. The Base model’s default end-of-sequence token is `<|endoftext|>`, whereas the chat template used in SFT training data employs `<|im_end|>` as the sequence delimiter. This mismatch causes the model to fail to recognize when a response should terminate, resulting in repetitive generation. To resolve this, we replace the Base model’s `generation_config.json` and `tokenizer_config.json` with those from Qwen2.5-3B-Instruct, which properly defines `<|im_end|>` as the end-of-sequence token. This configuration swap ensures that the model can correctly identify sequence boundaries during both SFT and subsequent RL training, without changing any model weights.

B Training Hyperparameters

Table A1 lists the hyperparameters used in our experiments. All RL variants share the same base configuration (group size, learning rate, batch size, training steps), and differ only in algorithm-specific parameters. For GRPO, we set the KL loss coefficient to 0.01 and the clipping ratio to 0.2. For GSPO, we use a tight clipping range with `clip_ratio_low` = 3×10^{-4} and `clip_ratio_high` = 4×10^{-4} , and no KL penalty. For GMPO, we set the KL loss coefficient to 0.001 and the clipping ratio to 0.4.

Table A1: Detailed training hyperparameters.

SFT	
Base model	Qwen2.5-3B-Base
SFT dataset	Wireless-CoT-Mix (3,542 examples)
Max response length	768
Learning rate	3e-6
Batch size	36
Epochs	1
Hardware	4090*3
RL (shared)	
RL dataset	WirelessMATHBench-XL train split
Evaluation set	WirelessMATHBench-XL test split
RL algorithms	GRPO, GSPO, GMPO
Group size	4
Max response length	768
Learning rate	1e-6
Batch size	18
Training steps	1000
Hardware	4090*3

C Extended Benchmark Comparison

Table A2 provides an extended comparison with proprietary models, open-source general-purpose models, and open-source math-specialized models. We include the complete set of reported metrics to place the SignalReasoner experiments in a broader context.

The extended results reveal several patterns. First, the strongest proprietary and open-source general-purpose models achieve substantially higher overall accuracy than the 3B SignalReasoner variants. GPT-5 obtains the highest overall score among the listed external models at 57.87%, while DeepSeek-R1 reaches 57.37% and DeepSeek-V3.1 reaches 56.87%. This gap is expected given the considerable differences in model scale and general mathematical capability, but it also highlights the difficulty of signal-domain mathematical reasoning for compact models.

Second, the SignalReasoner variants remain competitive with several smaller open-source baselines. B-GMPO achieves the highest overall accuracy among our 3B models at 39.12%, exceeding Qwen2.5-7B-Instruct (25.75%), Gemma 3 27B (31.50%), and all three listed math-specialized baselines at 7B or 72B except Qwen2.5-Math-72B-Instruct (42.13%). The category-level results also show that improvements are not limited to one question type: B-GMPO obtains the strongest SignalReasoner Fill-in score at 38.03% and the strongest FEC score at 31.41%, whereas A-GSPO obtains the highest MCQ score at 61.65%.

Finally, the output-token statistics show a clear efficiency trade-off among the reinforcement-learning variants. B-GSPO produces the shortest average response, 243.59 tokens, while still reaching 37.75% overall accuracy. B-GMPO is slightly longer at 258.87 tokens but achieves the best overall accuracy. In contrast, the GRPO variants generate substantially longer responses, with 668.33 and 600.10 tokens for A-GRPO and B-GRPO, respectively. These results support the observation that sequence-level optimization can improve response concision while preserving, and in the case of GMPO improving, answer accuracy.

Table A2: Extended comparison on the WirelessMATHBench-XL test set (%).

Model	Size	MCQ	Fill-in	FEC	Overall	Output Tokens
<i>Proprietary Models</i>						
GPT-5	–	63.91	63.20	41.36	57.87	–
GPT-5-mini	–	67.67	53.99	40.31	53.00	–
GPT-5-nano	–	57.14	37.82	30.37	39.25	–
GPT-4o	–	54.14	43.62	24.61	40.37	–
o4-mini	–	67.67	49.56	40.31	50.38	–
Claude-4.0-Sonnet	–	60.15	56.30	42.93	53.75	–
Gemini-2.5-Flash	–	63.16	56.09	43.46	54.25	–
Gemini-2.5-Pro	–	66.17	50.42	36.65	49.75	–
Grok-4-Fast	–	70.31	56.33	40.33	54.89	–
<i>Open-Source General Models</i>						
DeepSeek-R1	671B	65.41	60.50	43.98	57.37	–
DeepSeek-V3.1	671B	66.17	58.85	45.03	56.87	–
Llama-3.3-70B-Instruct	70B	54.14	38.03	28.27	38.37	–
Qwen2.5-72B-Instruct	72B	51.88	35.50	32.46	37.50	–
Qwen2.5-7B-Instruct	7B	39.10	21.85	26.18	25.75	–
Gemma 3 27B	27B	42.11	30.04	27.75	31.50	–
Gemma 3 12B	12B	36.84	21.43	21.99	24.12	–
<i>Open-Source Math-Specialized Models</i>						
Qwen2.5-Math-72B-Instruct	72B	60.15	40.55	33.51	42.13	–
Qwen2.5-Math-7B-Instruct	7B	42.11	14.71	24.61	21.62	–
DeepSeekMath-7B-RL	7B	43.61	13.66	25.65	21.50	–
<i>SignalReasoner 3B Models</i>						
Base	3B	28.57	9.03	9.42	12.37	907.41
Base-SFT	3B	42.11	26.68	24.61	28.75	604.45
A-GRPO	3B	57.89	30.88	27.23	34.50	668.33
A-GSPO	3B	61.65	30.88	26.18	34.88	279.25
A-GMPO	3B	51.88	32.98	26.70	34.63	395.84
B-GRPO	3B	49.62	33.61	25.65	34.38	600.10
B-GSPO	3B	57.89	35.08	30.37	37.75	243.59
B-GMPO	3B	54.14	38.03	31.41	39.12	258.87